

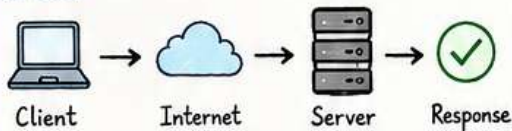
Top 50 Backend Interview Concepts

Backend Fundamentals

① Client-Server Architecture

A distributed architecture where **clients** request services or resources from a **server**. The server processes the request and sends back a **response**.

Flow:



- **Client** : Browser / Application
- **Server** : Business logic + Data
- Client sends a request.
- **Server** processes the request and sends a response.

② REST API

REST (Representational State Transfer) is an architectural style for building APIs using the **HTTP** protocol.

Key Points:

- ★ Stateless
- ★ Client-Server
- ★ Cacheable
- ★ Layered System
- ★ Uniform Interface
- ★ Code on Demand (Optional)

③ HTTP Methods

Method	Purpose	Ideal Use
GET	Retrieve a resource	Read data
POST	Create a new resource	Create data
PUT	Update an existing resource fully	Replace data completely
PATCH	Update an existing resource partially	Modify data partially
DELETE	Delete a resource	Remove data

④ HTTP Status Codes

Common Codes:

- 1xx → Informational
 - 2xx → Success
 - 3xx → Redirection
 - 4xx → Client Error
 - 5xx → Server Error
- 200 OK → Request successful
 - 201 Created → Resource created
 - 400 Bad Request → Invalid request
 - 401 Unauthorized → Authentication required
 - 403 Forbidden → Access denied
 - 404 Not Found → Resource not found
 - 500 Internal Server Error → Server issue

⑤ Stateless vs Stateful

Stateless

- Server does not store client context.
- Each request contains all necessary data.
- Example: JWT / Token-based authentication
- Scalable and reliable.

Stateful

- Server stores client session/context.
- Example: Session ID, Cart stored on server
- Harder to scale.
- Commonly used in traditional applications.



Tip: Most REST APIs are stateless.

⑥ Authentication vs Authorization

Authentication



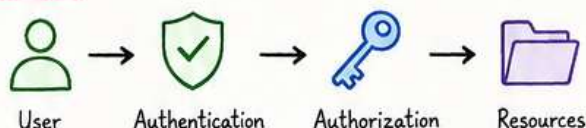
- Verifies who you are.
- Related to login and identity.

Authorization



- Verifies what you can do.
- Related to permissions and roles.

Flow:

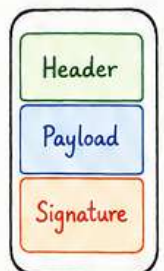


⑦ JWT (JSON Web Token)

JWT is a compact, URL-safe token used for securely transmitting information between parties.

JWT consists of three main parts:

- ① **Header** → Contains token information and algorithm.
- ② **Payload** → Contains claims/information.
- ③ **Signature** → Used to verify the token's integrity using a secret key.



JWT

JWT is used for:

- Authentication
- Information exchange
- Stateless sessions



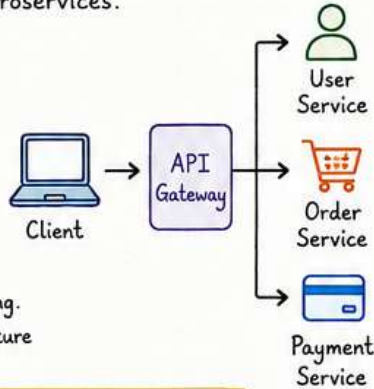
API & Backend Design

⑧ API Gateway

An API Gateway is a single entry point for all client requests in a system with multiple services or microservices.

Main Functions:

- Acts as a single entry point for client requests.
- Routes requests to the appropriate services.
- Handles authentication.
- Performs rate limiting.
- Provides logging and caching.
- Hides the internal architecture from clients.



Tip: Use an API Gateway when you have multiple services or microservices.

⑨ Rate Limiting

Rate Limiting limits the number of requests a client can make within a given time window.

Purpose:

- Prevents abuse.
- Protects the system from excessive requests.
- Ensures fair usage.
- Can be implemented at the API Gateway or service level.



Example:

100 requests per IP per minute
Time Window = 60 seconds

⑩ Idempotency

Idempotency means that making multiple identical requests has the same effect as making the request once.

- Important for **POST/PUT** requests, especially operations that may be retried.
- Use an **Idempotency-Key** header to ensure request uniqueness.

Use Cases:

- Payment requests
- Order creation
- Retry mechanisms

Example:

If a payment request is accidentally sent twice with the **same idempotency key**, the server should avoid creating two payments.

⑪ Pagination

Pagination means breaking a large result set into smaller pages.

Benefits:

- Improves performance.
- Reduces response size.
- Improves user experience.
- Makes it easier to handle large datasets.

Common Types:

- 1 Page Number Based
- 2 Cursor Based
- 3 Offset Based

Response:

```
{
  "data": [...10 users...],
  "page": 2,
  "limit": 10,
  "totalPages": 25,
  "totalItems": 248
}
```

Example:

GET /api/users?page=2&limit=10

⑫ API Versioning

API Versioning is used to manage API changes without breaking existing clients.

Versions can be specified using:

- URL
- Header
- Query Parameter

Example:

/api/v1/users
/api/v2/users

★ **Best Practice:** Add new versions when major changes are required, but do not break existing versions.

⑬ Webhooks

A Webhook is a server-to-server callback used to notify another system when an event occurs.

Features:

- Server-to-server communication.
- Sends real-time event notifications.
- Commonly uses an **HTTP POST** request.

Common Uses:

- Payment systems
- CI/CD
- CRM
- Order notifications

Example:



Real-world Example:

A payment gateway sends a webhook to your application after a payment succeeds or fails.

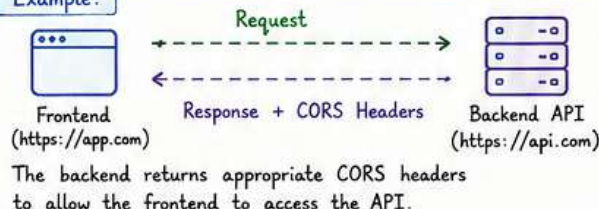
⑭ CORS

CORS (Cross-Origin Resource Sharing) is a mechanism that allows restricted resources on a web page to be requested from another domain. It is controlled by the server using HTTP headers.

Purpose:

- Allows controlled cross-origin requests.
- Helps secure communication between different origins.

Example:



Common CORS Headers:

- Access-Control-Allow-Origin
- Access-Control-Allow-Methods
- Access-Control-Allow-Headers

Database Concepts

15 SQL vs NoSQL

	SQL (Relational)	NoSQL (Non-Relational)
	Table-based (Rows, Columns)	Key-Value / Document / Column / Graph
	Fixed schema	Flexible schema
	ACID compliant	BASE (Eventually consistent)
	Joins supported	Joins are not common
	Vertically scalable	Horizontally scalable
	Examples: MySQL, PostgreSQL	Examples: MongoDB, Cassandra, Redis



Remember:

- Use **SQL** for complex relationships.
- Use **NoSQL** for flexible and high-scale requirements.

16 Indexing

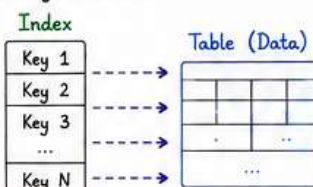
Indexing improves database query performance by reducing search time. An index is a data structure that helps the database find rows faster.

Types of Indexes

- ① **Primary Index** → Created on the Primary Key.
- ② **Unique Index** → Ensures unique values.
- ③ **Composite Index** → Created using multiple columns.
- ④ **Full-Text Index** → Used for searching text data.

How Index Works

The index stores keys that point to the corresponding data in the table.



Tip: Indexing speeds up **READ** operations

but may slow down **WRITE** operations because indexes also need to be updated.

17 Transactions & ACID

A **transaction** is a sequence of one or more SQL operations treated as a single unit of work.

ACID Properties

- A Atomicity**
 - All or nothing.
 - Either the complete transaction succeeds or the entire transaction is rolled back.
- C Consistency**
 - Database remains in a valid state before and after the transaction.
- I Isolation**
 - Concurrent transactions do not interfere with each other.
- D Durability**
 - Once a transaction is committed, changes persist even after a system failure.

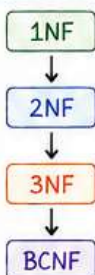
19 Database Normalization

Database Normalization is the process of organizing data to reduce redundancy and improve data integrity.

Normal Forms

- 1NF** - First Normal Form
 - Values must be atomic.
 - No repeating groups.
- 2NF** - Second Normal Form
 - Must satisfy 1NF.
 - No partial dependency.
- 3NF** - Third Normal Form
 - Must satisfy 2NF.
 - No transitive dependency.
- BCNF** - Boyce-Codd Normal Form
 - A stronger version of 3NF.

Normalization Flow



18 Isolation Levels

Isolation levels control how transactions behave when multiple transactions run concurrently.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	✓	✓	✓
Read Committed	✗	✓	✓
Repeatable Read	✗	✗	✓
Serializable	✗	✗	✗

① Read Uncommitted

- Highest concurrency.
- Lowest isolation.
- Dirty reads are possible.

② Read Committed

- Prevents dirty reads.
- Non-repeatable and phantom reads may occur.

③ Repeatable Read

- Prevents dirty reads.
- Prevents non-repeatable reads.
- Phantom reads may occur.

④ Serializable

- Highest isolation level.
- Transactions run serially.
- Prevents dirty, non-repeatable, and phantom reads.

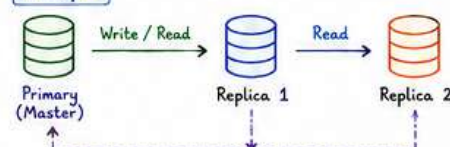
20 Replication

Replication means copying data from one database server to another database server.

Benefits

- Improves availability.
- Improves reliability.
- Helps with read scaling.
- Provides redundancy.

Example



Acknowledgement is sent between the database servers to confirm replication.

21 Sharding

Sharding means splitting data across multiple database servers called **shards**.

Benefits

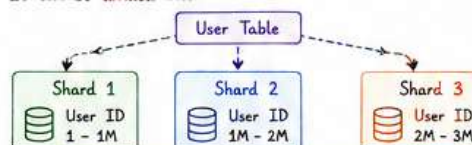
- Improves read scalability.
- Improves write scalability.
- Allows large datasets to be distributed across multiple servers.

Types of Sharding

- ① **Horizontal Sharding**
 - Data is divided by rows.
- ② **Vertical Sharding**
 - Data is divided by columns.

Horizontal Sharding Example

Suppose a large **User** Table contains millions of users. It can be divided into:



The **Sharding Key** (e.g., User ID) determines which shard stores the data.

★ Remember

- ★ **SQL** → Complex relationships.
- ★ **NoSQL** → Flexible schema and high-scale needs.
- ★ **Good Schema + Index** → High performance.

20. TOP 50 BACKEND INTERVIEW CONCEPTS

Caching & Performance (22-28)

22 Cache

Cache stores frequently used data in memory for fast access.

Benefits:

- Reduces database load.
- Improves response time.

Types:

- In-Memory Cache
- Browser Cache
- CDN Cache

Examples: Redis, Memcached

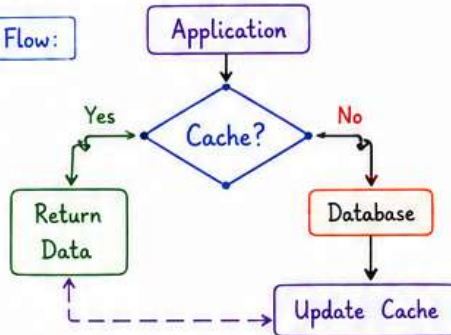
Tip: Always set proper TTL (Time To Live). Don't cache everything!

23 Cache-Aside Pattern (Lazy Loading)

Steps:

- 1 Application looks for data in cache.
- 2 If found → Return data.
- 3 If not found → Fetch from database.
- 4 Store data in cache.
- 5 Return data to client.

Flow:



24 Write-Through vs Write-Back

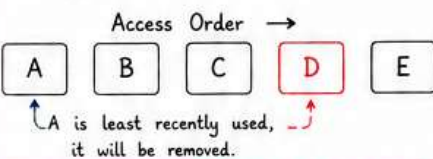
Write-Through	Write-Back
Writes to cache and DB together	Writes to cache first
More consistent	Faster writes
Slower writes	DB updated later
Lower risk of data loss	Risk of data loss
Banking, Orders	Logs, Analytics

25 Cache Eviction Policies

- ☆ LRU → Least Recently Used
- ☆ LFU → Least Frequently Used
- ☆ FIFO → First In First Out
- ☆ TTL → Time To Live
- ☆ Random → Randomly removes items

LRU Example:

When cache is full, the least recently used item is removed.



26 Redis

Redis is an in-memory data structure store.

Supports:

- Strings ("abc")
- Hashes {key: value}
- Lists [a, b, c]
- Sets {a, b, c}
- Sorted Sets {1, 2, 3}

Used For:

- Caching
- Sessions
- Rate limiting
- Leaderboards

Features:

- Persistence
- Pub/Sub
- Replication
- TTL

Common Commands:

SET key value	# Set key	HSET user:1 name abhi	# Set hash field
GET key	# Get value	HGET user:1 name	# Get hash field
DEL key	# Delete key	LPUSH list a	# Push to list
EXPIRE key 60	# Set TTL (60 sec)	LRANGE list 0 -1	# Get all items

27 CDN (Content Delivery Network)

A CDN distributes content across multiple edge servers.

Benefits:

- Delivers content closer to users.
- Improves load time.
- Improves availability.
- Reduces load on origin server.

Flow:



If content exists in CDN → Serve it.
If not → Fetch from origin server.

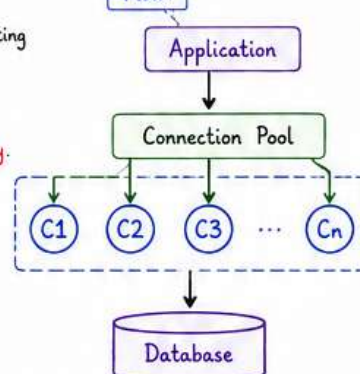
28 Connection Pooling

Connection pooling reuses existing database connections.

Benefits:

- Avoids overhead of creating new connections.
- Improves performance.
- Handles high concurrency.
- Configurable pool size.

Flow:



How it works:

Connections are reused from the pool instead of creating new connections every time.

BACKEND INTERVIEW HANDBOOK

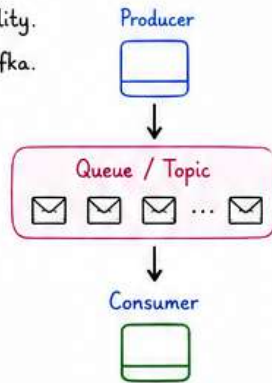
PAGE 5 — ASYNC & DISTRIBUTED SYSTEMS (29-35)

29 Message Queue

- Enables systems to communicate asynchronously.
- Decouples producer and consumer.
- Improves reliability and scalability.
- Examples: RabbitMQ, SQS, Kafka.

Benefits:

- ★ Async Processing
- ★ Load Leveling
- ★ Fault Tolerance
- ★ Better Scalability

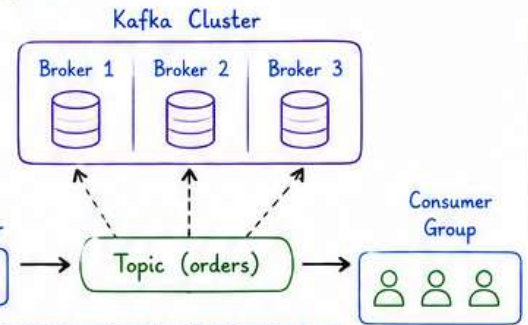


30 Kafka

- Distributed event streaming platform.
- High throughput, scalable, fault tolerant.
- Stores events in topics.

Key Components:

- ★ Producer
- ★ Broker
- ★ Topic
- ★ Consumer



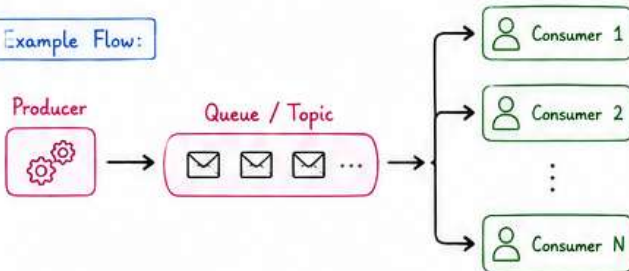
Kafka Flow:

Producer → Topic → Consumer Group

31 Producer / Consumer

- Producer:** Sends messages to the queue/topic.
- Consumer:** Reads and processes messages.
- Multiple consumers can read for parallel processing.
- Ensures high availability and scalability.

Example Flow:

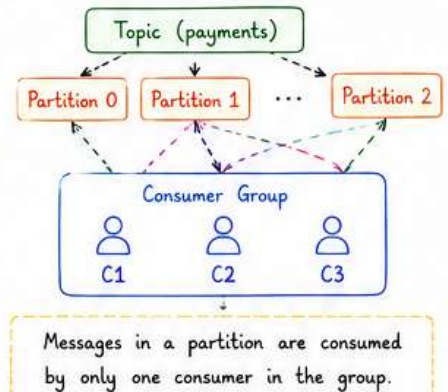


32 Consumer Groups

- Consumers are grouped to share the load.
- Each message is consumed by only one consumer in the group.
- Enables horizontal scaling of consumers.

Example:

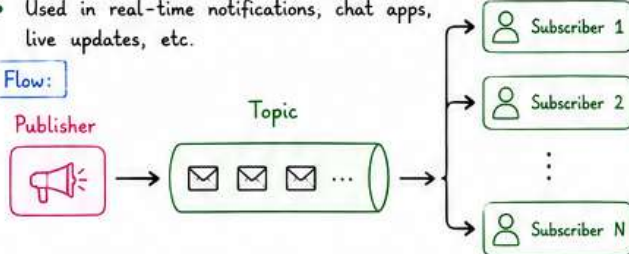
- Topic → Consumer Group
- C1, C2, C3
- Each partition → one consumer
- Ensures parallel consumption.



33 Pub / Sub (Publish / Subscribe)

- Publisher** sends messages to a topic.
- Subscribers** receive messages from the topic.
- One-to-many communication model.
- Used in real-time notifications, chat apps, live updates, etc.

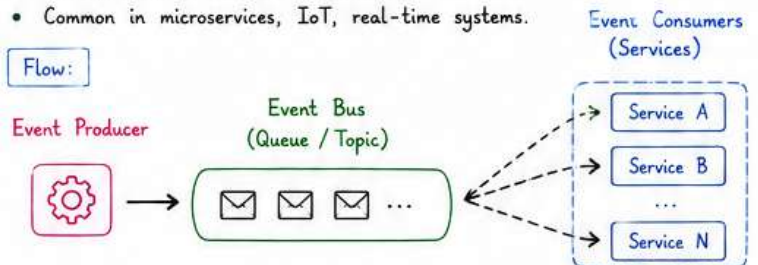
Flow:



34 Event-Driven Architecture

- System reacts to **events**.
- Components communicate via events.
- Loosely coupled, scalable and resilient.
- Common in microservices, IoT, real-time systems.

Flow:



35 Synchronous vs Asynchronous

Synchronous	Aspect	Asynchronous
Request → Wait → Response	Flow	Request → Continue → Later Response
Blocking call	Nature	Non-blocking call
Lower throughput	Throughput	Higher throughput
Tight coupling	Coupling	Loose coupling
Example: REST API call	Example	Example: Message Queue, Kafka

★ Interview Tips

- Understand the trade-offs between **consistency** and **availability**.
- Know when to use **Queue** vs **Kafka**.
- Event-driven systems are key for building **scalable architectures**.
- Focus on **fault tolerance** and **idempotency** in async systems.

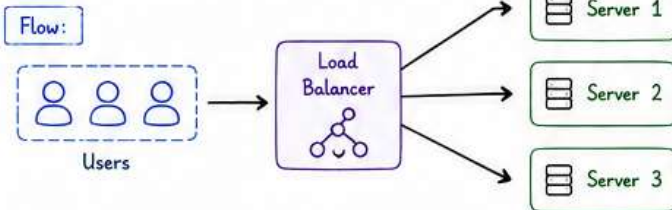


BACKEND INTERVIEW HANDBOOK

PAGE 6 — SCALABILITY & RELIABILITY (36-43)

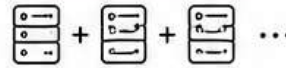
36 Load Balancer

- Distributes incoming traffic across multiple servers.
- Improves availability and prevents any single server from being overwhelmed.
- Types:
 - L4 → Transport Layer (e.g., TCP, UDP)
 - L7 → Application Layer (e.g., HTTP, HTTPS)



37 Horizontal vs Vertical Scaling

Horizontal Scaling (Scale Out)



- Add more machines/servers.
- Handles more traffic.
- Cost-effective.

Vertical Scaling (Scale Up)



- Add more power to an existing machine.
- Limited by hardware.

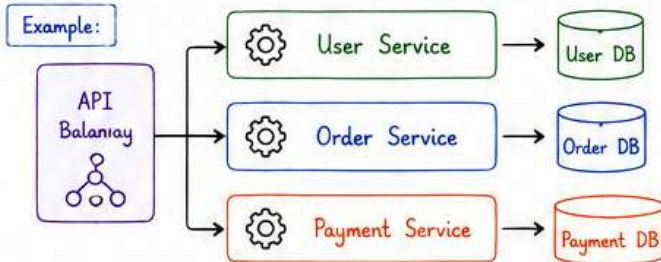


Horizontal → Add more servers

Vertical → Increase server power

38 Microservices

- Application is broken down into small, independent services.
- Each service can have its own database.
- Improves scalability, flexibility, and team productivity.
- Services communicate through APIs.

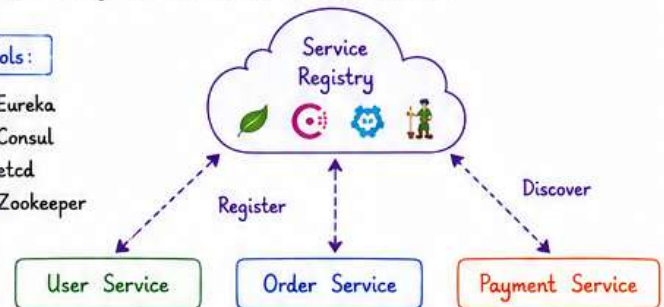


39 Service Discovery

- Mechanism to find and communicate with services automatically.
- Removes hard-coded service locations.
- Useful in dynamic environments like microservices.

Tools:

- Eureka
- Consul
- etcd
- Zookeeper



40 Circuit Breaker

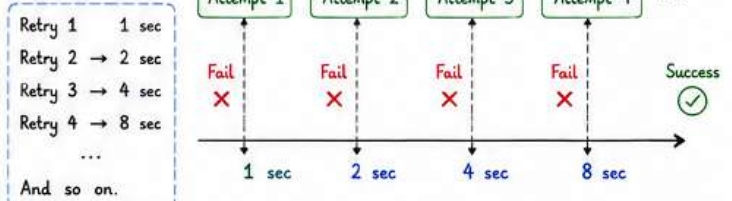
- Prevents cascading failures by stopping calls to failing services.
- States:
 - Closed → Normal operation
 - Open → Fail fast (no calls allowed)
 - Half-Open → Test calls (limited calls allowed)
- Then back to Closed if successful.



41 Retry & Exponential Backoff

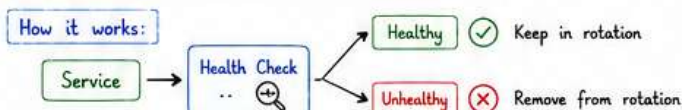
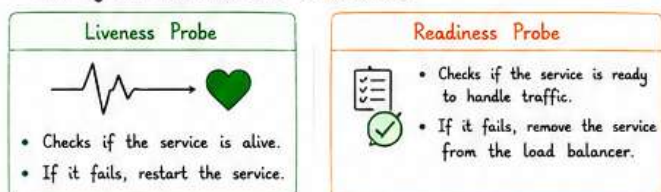
- Retries failed requests to handle temporary/transient errors.
- Exponential backoff increases the waiting time between retries.
- Prevents overwhelming downstream services.

Example:



42 Health Checks

- Periodic checks to ensure services are running properly.
- Used by load balancers and orchestrators.



43 Observability

- Ability to understand the internal state of systems.

Three Pillars:



Interview Tips:

- Understand the trade-offs between consistency and availability.
- Know when to use Queue vs Kafka.

- Event-driven systems are key for building scalable architectures.
- Focus on fault tolerance and idempotency in async systems.

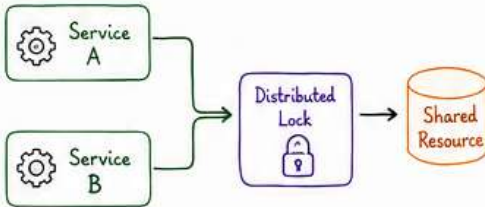
BACKEND INTERVIEW HANDBOOK

PAGE 7 — ADVANCED CONCEPTS & CHEAT SHEET (44-50)

44 Distributed Lock

- Ensures only one process can access a shared resource at a time.
- Prevents race conditions in distributed systems.
- Commonly implemented using Redis, ZooKeeper, etc.

Flow:

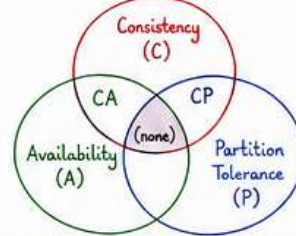


45 CAP Theorem

- In a distributed system, you can achieve only two out of three:
- C → Consistency
- A → Availability
- P → Partition Tolerance

Remember:

CAP → Choose any 2



46 Consistency Models



Strong Consistency

- All reads return the latest write.



Eventual Consistency

- System may be temporarily inconsistent.
- System becomes consistent over time.



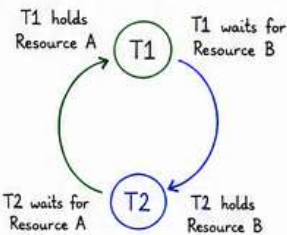
Causal Consistency

- Guarantees ordering of causally related operations.

47 Database Deadlocks

- Occurs when two or more transactions wait for each other indefinitely.
- Database detects and resolves the deadlock by aborting one transaction.

Deadlock Example:



Prevention Tips:

- Keep transactions short.
- Access tables in the same order.
- Use proper indexing.
- Avoid long-running locks.

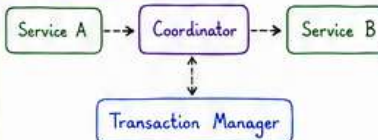
48 Distributed Transactions

- Transactions that span across multiple services or databases.
- Ensures ACID across systems.

Common Protocols:

- ★ 2PC (Two-Phase Commit)
- ★ Saga Pattern
- ★ TCC (Try-Confirm-Cancel)

Example Flow:



49 Eventual Consistency

- System may not be immediately consistent.
- It will become consistent over time.
- Improves availability and partition tolerance.

Examples:

- DynamoDB
- Cassandra
- Riak

Example:



Initially, Node B may have old data. After some time, both nodes become synchronized.



BACKEND INTERVIEW RAPID REVISION



1. REST

- Stateless and scalable API architecture.
- Use proper HTTP methods and status codes.

Methods:
GET, POST, PUT,
PATCH, DELETE



Status Codes:
200, 201, 400, 401,
404, 500

2. Redis (Cache)

- In-memory key-value store.
- Improves performance.
- Use Cache-Aside Pattern.



Key → Value
user:1 → { name: "abhi" }

3. Kafka (MQ)

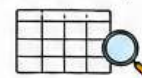
- Distributed event streaming platform.
- High throughput.
- Decouples services.



Producer → Topic → Consumer

4. Database Index

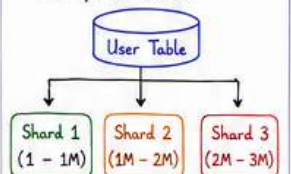
- Speeds up read/query operations.
- Use proper indexes.
- Avoid over-indexing.



CREATE INDEX idx_user_id
ON users(id);

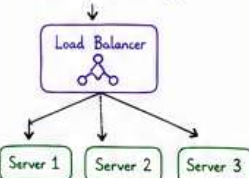
5. Sharding

- Horizontal scaling by splitting data across multiple databases.



6. Load Balancer

- Distributes traffic across servers.
- Improves availability.



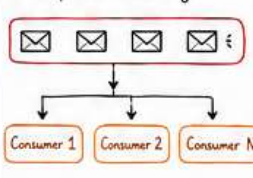
7. Circuit Breaker

- Protects against cascading failures.
- Improves system stability.



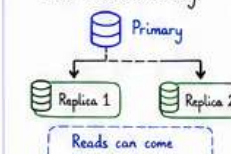
8. Message Queue

- Decouples services.
- Handles traffic spikes.
- Improves reliability.



9. Replication

- Copies data to multiple nodes.
- Improves availability and read scalability.



10. CAP Theorem

- Choose only two of Consistency, Availability, and Partition Tolerance.



★ INTERVIEW TIP

When explaining any backend concept, try to cover these 5 points:

1. Definition



What is it?
Define the concept in simple words.

2. Why it is needed



What problem does it solve?
Why do we need it?

3. How it works



Explain the working or flow of the concept.

4. Pros / Cons



Advantages and limitations of the concept.

5. Real-world example



Where is it used?
Give real-world examples.